

SU/FS/master/info/4I503 APS

Examen réparti 2

Mai 2019

On se place dans le cadre de APS2 pour y ajouter quelques extensions.

Extension 1: dans le langage PASCAL, à côté de la boucle `while e do cs`, il y a une boucle `repeat cs until e`. Les différences entre ces deux boucles sont les suivantes:

<code>while e do cs</code>	<code>repeat cs until e</code>
test <i>e</i> avant <i>cs</i> sortie si faux	test <i>e</i> après <i>cs</i> sortie si vrai

Exemple (dans la syntaxe PASCAL)

- `var x:Integer; x:=0; while (0 < x) begin write(42); end;` ne produit rien sur le flot de sortie;
 - `var x:Integer; x:=0; repeat write(42); until (0 < x);` produit indéfiniment 42 sur le flot de sortie.
1. proposez une *syntaxe concrète* pour cette nouvelle instruction.
 2. donnez l'extension de syntaxe nécessaire.
 3. ajouter les règles de typage pour cette instruction.
 4. définissez les règles sémantiques de cette instruction.

Extension 2: on souhaite ajouter une affectation multiple et parallèle. On propose la syntaxe concrète suivante:

`MSET ( $x_1, \dots, x_n$ ) ( $e_1, \dots, e_n$ )`

où les $x_1, \dots, x_n$ sont des identificateurs et $e_1, \dots, e_n$ des expressions.

Cette affectation est «parallèle» en ce sens que le programme

```
[  
  VAR x int;  
  VAR y int;  
  SET x 2;  
  SET y 4;  
  MSET (x,y) (y,x);  
  ECHO x; ECHO y;  
]
```

produira sur le flot de sortie la suite 42.

1. donnez l'extension de syntaxe pour ajouter cette nouvelle instruction.
2. ajouter les règles de typage pour cette instruction.
3. définissez les règles sémantiques de cette instruction.

Extension 3a: le noyau fonctionnel du langage APS souffre d'une lacune: on ne peut pas définir de constante locale à une expression. Par exemple pour calculer la valeur de $ax^3 + bx^2 + cx + d$ on aimerait calculer x^2 une seule fois ($x^3 = x^2x$) en écrivant (syntaxe ML) `let xx = x*x in a*xx*x + b*xx + c*x + d`.

On rappelle que la construction `let-in` en ML produit une expression. Par exemple, on peut écrire (syntaxe ML) `(let x = 7 in x+14) * 2` qui a pour valeur 42.

1. donnez l'extension de syntaxe nécessaire pour cette expression.
2. ajoutez la règle de typage de cette nouvelle expression.
3. définissez la règle sémantique d'évaluation de cette expression.

Extension 3b: l'extension précédente permet d'avoir des fonctions locales à une expression, puisque l'on a des expressions fonctionnelles. On a donc la possibilité de définir des fonctions locales, en particulier, dans des définitions de fonctions. Mais il est souvent utile d'avoir en plus la possibilité de définir des fonctions locales *récurrentes*.

Exemple d'utilisation d'une fonction récursive locale (syntaxe ML):

```
let fac (n:int) : int =  
  let rec loop n r =  
    if (n > 0) then (loop (n-1) (n*r))  
    else r  
  in  
    (loop n 1)
```

L'extension 3a ne permet pas cela car une expression fonctionnelle ne peut pas être récursive.

1. donnez l'extension de syntaxe nécessaire pour la définition locale d'une fonction récursive.
2. ajoutez la règle de typage de cette nouvelle expression.
3. définissez la règle sémantique d'évaluation de cette expression.